ShowerShapesAnalysis

Software Documentation

Photon Shower Shape Reweighting — MC23e

Version 1.0

ATLAS Simulation

February 24, 2026

Contents

1 Overview **3**

2 Software Requirements **3**
- 2.1 Operating system . . . 3
- 2.2 Required software . . . 3
- 2.3 Python libraries . . . 4

3 Installation **4**
- 3.1 Obtaining the repository . . . 4
- 3.2 Environment setup . . . 4
- 3.3 Compilation . . . 4

4 Repository Structure **5**
- 4.1 Validation scripts (`scripts/`) . . . 5
- 4.2 Closure test pipeline (`scripts/closure_test/`) . . . 5
- 4.3 Grid submission (`grid/`) . . . 5

5 Input Data **5**
- 5.1 Expected directory layout . . . 5
- 5.2 Ntuple format . . . 7
- 5.3 Sample details . . . 7

6 Running the Analysis **7**
- 6.1 Shower shape validation . . . 8
  - 6.1.1 Step 1: Compute histograms . . . 8
  - 6.1.2 Step 2: Generate plots . . . 8
- 6.2 Closure test . . . 9
  - 6.2.1 Automated execution . . . 9
  - 6.2.2 Command-line parameters . . . 9
  - 6.2.3 Manual step-by-step execution . . . 10
  - 6.2.4 Distortion model . . . 11
  - 6.2.5 Correction methods . . . 11

6.2.6 Quality cuts 12
6.2.7 Configuration parameters (`config.h`) 12
6.3 Grid submission (ntuple production) 12

7 Output **12**
7.1 Validation output 13
7.2 Closure test output 13

8 Reproducibility **14**
8.1 Full reproduction from scratch 14
8.2 Expected runtime 15
8.3 Random seeds 15
8.4 Version control 15

1 Overview

`ShowerShapesAnalysis` is a ROOT/Python analysis package for studying and correcting photon electromagnetic shower shape variables using cell-level calorimeter information from ATLAS MC23e simulated ntuples.

The package provides two main capabilities:

1. **Shower shape validation**: Recomputes the shower shape variables R_η, R_ϕ, and $w_{\eta 2}$ directly from Layer-2 cell energies and compares them against the stored (fudged and unfudged) values in the ntuples. This validates the cell-level computation.

2. **Cell-energy reweighting closure test**: A complete pipeline that generates pseudo-data by distorting MC cell energies, derives per-cell reweighting corrections, applies them, and evaluates how well the corrected MC recovers the pseudo-data distributions. Two correction methods are implemented and compared.

The package is designed to run on CERN lxplus (EL9) with ROOT 6.32+ available via the ATLAS software distribution on CVMFS. It operates on ntuples produced by the `NTupleMaker` Athena algorithm, which stores the 7×11 Layer-2 cell energies for each photon candidate.

2 Software Requirements

2.1 Operating system

The package is developed and tested on CERN lxplus nodes running AlmaLinux 9 (EL9). It should work on any Linux system with access to CVMFS.

2.2 Required software

Table 1: Software dependencies.

Software	Version	Purpose
ROOT	≥ 6.32	Event processing, histogramming, plotting, C++ JIT compilation via `gInterpreter`
Python	3.9+	Shower shape validation scripts (bundled with ROOT distribution)
Athena	25.0.40	NTupleMaker compilation (ntuple production only)
pathena	(via `lsetup panda`)	Grid job submission (ntuple production only)
rucio	(via `lsetup rucio`)	Dataset management (ntuple production only)

Note. Athena, pathena, and rucio are required only for producing the input ntuples from AOD files on the grid. If pre-produced ntuples are already available, only ROOT 6.32+ is needed.

2.3 Python libraries

The Python scripts use only `ROOT` (PyROOT) and `numpy`, both of which are included in the ROOT distribution available on CVMFS. No additional `pip install` is required.

3 Installation

3.1 Obtaining the repository

Clone the repository from GitHub:

```
git clone git@github.com:martafsilva31/ShowerShapesAnalysis.git
cd ShowerShapesAnalysis
```

3.2 Environment setup

On lxplus, source the provided setup script. This configures the ATLAS software environment (Athena + ROOT) and optionally sources the NTupleMaker build:

```
# Navigate to the parent ShowerShapes directory
cd /path/to/ShowerShapes/
source ShowerShapesAnalysis/setup.sh
```

The setup script performs the following steps:

1. Sets environment variables `SHOWERSHAPES_DIR` and `ANALYSIS_DIR`.
2. Runs `setupATLAS` and `asetup Athena,25.0.40`.
3. Sources the NTupleMaker build (if present).
4. Defines convenience aliases (`cdanalysis`, `cdntuples`, etc.).

Minimal setup (analysis only). If ntuples are already available and no Athena build is needed, a lightweight alternative is:

```
setupATLAS
lsetup "root 6.32.08-x86_64-el9-gcc13-opt"
```

3.3 Compilation

No compilation step is required. The C++ ROOT macros are compiled at runtime via ROOT's JIT compiler (`gInterpreter`). The closure test macros include `config.h` as a shared header, which ROOT processes automatically when the macro is invoked.

Table 2: Top-level directory layout.

Path	Description
`scripts/`	Analysis scripts (Python + ROOT C++ macros)
`scripts/closure_test/`	Closure test pipeline (6 C++ macros + shell drivers)
`grid/`	Grid submission scripts for ntuple production via pathena
`grid/samples/`	Sample lists (dataset container names)
`configs/`	Configuration files (reserved for future use)
`ntuples/`	Input ntuples (gitignored; see section 5)
`output/`	All generated output: histograms, plots, closure test results (gitignored)
`report/`	LaTeX reports and compiled PDFs
`setup.sh`	Environment setup script
`.github/`	Copilot instruction files for AI-assisted development

4 Repository Structure

4.1 Validation scripts (`scripts/`)

Table 3: Shower shape validation scripts.

File	Description
`utils.py`	Shared Python utilities: argument parsing, TTree I/O helpers, cell-energy window extraction, $w_{\eta 2}$ computation.
`compute_reta.py`	Recomputes $R_\eta = E(3 \times 7)/E(7 \times 7)$ from cell energies. Compares with stored unfudged and fudged values. Outputs histogram ROOT file. Uses JIT-compiled C++ for the inner loop.
`compute_rphi.py`	Same as above for $R_\phi = E(3 \times 3)/E(3 \times 7)$.
`compute_weta_2.py`	Same for $w_{\eta 2}$. Uses cluster η_2 plus relative cell offsets.
`plot_reta.C`	ROOT macro: reads histogram file, produces a single-page PDF with an overlay of the three distributions (unfudged, fudged, computed) and a ratio panel.
`plot_rphi.C`	Same layout for R_ϕ.
`plot_weta_2.C`	Same layout for $w_{\eta 2}$.

4.2 Closure test pipeline (`scripts/closure_test/`)

4.3 Grid submission (`grid/`)

5 Input Data

5.1 Expected directory layout

The analysis expects input ntuples under `ntuples/` with the following structure:

```
ntuples/
```

Table 4: Closure test source files and their roles.

File	Description
`config.h`	Shared header defining cluster geometry, $\|\eta\|$ binning (14 bins), energy-fraction binning (12 bins), branch names, quality-cut functions, shower shape formulas, and the distortion bias pattern.
`create_pseudodata.C`	Generates a pseudo-data ROOT file by applying systematic per-cell bias plus Gaussian noise to MC cell energies. Conserves total cluster energy.
`derive_corrections.C`	Derives reweighting corrections by comparing MC and pseudo-data cell fractions. Produces Method 1 (flat shift) and Method 2 (TProfile) corrections, stored in a single ROOT file.
`apply_corrections.C`	Reads corrections and applies them to MC cell energies. Outputs a reweighted ntuple. Accepts a method flag (1 or 2).
`validate_closure.C`	Fills shower shape histograms (R_η, R_ϕ, $w_{\eta 2}$) per $\|\eta\|$ bin for MC, pseudo-data, Method 1, and Method 2. Computes χ^2/n_{df} closure metrics.
`plot_closure.C`	Generates multi-page comparison PDFs (one page per $\|\eta\|$ bin) with overlay + ratio panels.
`run_closure_test.sh`	Driver script: runs all 6 pipeline steps for a single distortion level.
`run_closure_suite.sh`	Wrapper: runs `run_closure_test.sh` at 1%, 3%, and 5% distortion levels.

Table 5: Grid submission scripts.

File	Description
`submit_mc23e_Zllg.sh`	Submits pathena jobs for MC23e $Z \to \ell\ell\gamma$ samples (DSIDs 700770, 700771).
`submit_data22.sh`	Submits jobs for 2022 data runs.
`submit_data24.sh`	Submits jobs for 2024 data (single run).
`submit_data24_all.sh`	Submits jobs for all 2024 data runs.

```
mc23e/
  mc23e_700770_Zeeg.root        # Z -> eeg MC (DSID 700770)
  mc23e_700771_Zmumug.root      # Z -> mumug MC (DSID 700771)
data22/
  data22_XXXXXXXX_Zeeg.root     # Data 2022 runs
```

5.2 Ntuple format

The ntuples are produced by the `NTupleMaker` Athena algorithm and stored in a TTree named `tree`. The analysis uses the following branches:

Table 6: Required TTree branches (`tree`).

Branch name	Type	Description
`photon.7x11ClusterLr2E`	`vector<double>`	77 cell energies (MeV) in the $7\eta \times 11\phi$ Layer-2 window.
`photon.7x11ClusterLr2Size`	`int`	Number of cells (always 77 for valid clusters).
`photon_cluster.eta2`	`float`	Cluster barycentre pseudorapidity in Layer 2.
`photon.reta`	`float`	Stored R_η (fudged value).
`photon.unfudged_reta`	`float`	Stored R_η (unfudged value).
`photon.rphi`	`float`	Stored R_ϕ (fudged value).
`photon.unfudged_rphi`	`float`	Stored R_ϕ (unfudged value).
`photon.weta2`	`float`	Stored $w_{\eta 2}$ (fudged value).
`photon.unfudged_weta2`	`float`	Stored $w_{\eta 2}$ (unfudged value).

Cell energy layout. The 77 cell energies are arranged in a 7×11 ($\eta \times \phi$) grid. The flat index is calculated as:

$$\text{index} = \eta_{\text{row}} \times 11 + \phi_{\text{col}}, \qquad \eta_{\text{row}} \in [0, 6],\ \phi_{\text{col}} \in [0, 10]. \tag{1}$$

The central cell corresponds to $(\eta_{\text{row}}, \phi_{\text{col}}) = (3, 5)$, i.e. flat index 38.

5.3 Sample details

The primary MC sample used is:

- **DSID 700770**: `Sh_2214_eegamma` ($Z \to ee\gamma$), MC23e 13.6 TeV
- ∼9.6 million events, ∼13 GB
- AOD container: `mc23_13p6TeV.700770.Sh_2214_eegamma.merge.AOD.e8514_e8586_s4369_s437`

A secondary sample (DSID 700771, $Z \to \mu\mu\gamma$) is available for cross-checks.

6 Running the Analysis

All commands below assume the environment has been set up as described in section 3.

6.1 Shower shape validation

The validation workflow computes shower shape variables from cell energies and compares them against stored values. It consists of two steps per variable: a Python script that fills histograms, and a ROOT macro that produces comparison plots.

6.1.1 Step 1: Compute histograms

Run the Python compute scripts from the `scripts/` directory. Each script takes an input ntuple and produces a ROOT file containing histograms:

```
cd scripts/

# R_eta
python compute_reta.py \
  -i ../ntuples/mc23e/mc23e_700770_Zeeg.root \
  -o ../output/reta_Zeeg.root

# R_phi
python compute_rphi.py \
  -i ../ntuples/mc23e/mc23e_700770_Zeeg.root \
  -o ../output/rphi_Zeeg.root

# w_eta_2
python compute_weta_2.py \
  -i ../ntuples/mc23e/mc23e_700770_Zeeg.root \
  -o ../output/weta2_Zeeg.root
```

Repeat with the $Z \to \mu\mu\gamma$ ntuple (replace `mc23e_700770_Zeeg.root` with `mc23e_700771_Zmumug.ro` and adjust output names accordingly).

Parameters.

- `-i`, `--input`: Path to input ROOT ntuple (required).
- `-o`, `--output`: Path to output ROOT histogram file (required).
- `-n`, `--max-events`: Maximum number of events to process. Default: -1 (all events).

6.1.2 Step 2: Generate plots

Run the ROOT plotting macros. Each macro reads the histogram file from Step 1 and produces a single-page PDF:

```
# R_eta plot
root -l -b -q \
  'plot_reta.C("../output/reta_Zeeg.root","../output/plots/
     reta_Zeeg")'

# R_phi plot
root -l -b -q \
  'plot_rphi.C("../output/rphi_Zeeg.root","../output/plots/
     rphi_Zeeg")'
```

```
# w_eta_2 plot
root -l -b -q \
  'plot_weta_2.C("../output/weta2_Zeeg.root","../output/plots/
     weta2_Zeeg")'
```

The output argument is the base filename (without extension); the macro appends `.pdf` automatically. Create the output directory first if it does not exist:

```
mkdir -p ../output/plots
```

Each plot contains:

- **Upper panel**: Normalised distributions of three variants — Unfudged/stored (blue), Fudged/stored (red), Computed from cells (green dashed).
- **Lower panel**: Ratio to the Unfudged baseline (range 0.8–1.2).

6.2 Closure test

The closure test validates the cell-energy reweighting method using MC-only data. The pipeline consists of six sequential stages.

6.2.1 Automated execution

Single distortion level.

```
cd scripts/closure_test/

# Run with 500k events at 5% distortion
bash run_closure_test.sh 500000 0.05
```

Full suite (1%, 3%, 5%).

```
# Run all three distortion levels
bash run_closure_suite.sh 500000
```

For full-statistics runs using all ∼9.6M events:

```
bash run_closure_suite.sh -1
```

6.2.2 Command-line parameters

`run_closure_test.sh` accepts three positional arguments:

Table 7: Parameters for `run_closure_test.sh`.

Position	Name	Default	Description
1	`N_EVENTS`	500000	Number of events (-1 = all)
2	`DIST_LEVEL`	0.05	Systematic distortion magnitude
3	`NOISE_SIGMA`	-1	Per-cell noise sigma (-1 = auto: $\delta/2$)

Table 8: Parameters for `run_closure_suite.sh`.

Position	Name	Default	Description
1	`N_EVENTS`	500000	Events per level (-1 = all)

`run_closure_suite.sh` accepts one positional argument:
The suite always runs at distortion levels $\delta = 0.01, 0.03, 0.05$.

6.2.3 Manual step-by-step execution

Each step can be executed individually for debugging or partial reruns. All commands assume the working directory is `scripts/closure_test/` and use the following variables:

```
INPUT=../../ntuples/mc23e/mc23e_700770_Zeeg.root
OUTDIR=../../output/closure_test_5pct
LEVEL=0.05
N=500000
```

Step 1 — Create pseudo-data. Applies systematic per-cell distortion with Gaussian noise:

```
root -l -b -q \
  "create_pseudodata.C(\"$INPUT\",\"$OUTDIR/pseudo.root\",$N,
     $LEVEL)"
```

Step 2 — Derive corrections. Compares MC and pseudo-data cell fractions:

```
root -l -b -q \
  "derive_corrections.C(\"$INPUT\",\"$OUTDIR/pseudo.root\",\
  \"$OUTDIR/corrections.root\",$N)"
```

Step 3 — Apply Method 1 (flat shift).

```
root -l -b -q \
  "apply_corrections.C(\"$INPUT\",\"$OUTDIR/corrections.root\",\
  \"$OUTDIR/reweighted_m1.root\",$N,1)"
```

Step 4 — Apply Method 2 (TProfile).

```
root -l -b -q \
  "apply_corrections.C(\"$INPUT\",\"$OUTDIR/corrections.root\",\
  \"$OUTDIR/reweighted_m2.root\",$N,2)"
```

Step 5 — Validate (histograms + χ^2).

```
root -l -b -q \
  "validate_closure.C(\"$INPUT\",\"$OUTDIR\",\
  \"$OUTDIR/closure_histos.root\",$N)"
```

Step 6 — Plot.

```
root -l -b -q \
  "plot_closure.C(\"$OUTDIR/closure_histos.root\",\"$OUTDIR/\")"
```

6.2.4 Distortion model

The pseudo-data generation (`create_pseudodata.C`) applies the following transformation to each cell energy e_k:

$$e'_k = e_k \cdot \Big(1 + \delta \cdot b_k + \mathcal{G}(0, \sigma)\Big), \tag{2}$$

where δ is the distortion level, b_k is a fixed bias pattern based on the Chebyshev distance from the central cell (table 9), and $\mathcal{G}(0, \sigma)$ is per-event per-cell Gaussian noise with $\sigma = \delta/2$ by default. After scaling, total cluster energy is conserved:

$$e'_k \leftarrow e'_k \cdot \frac{E_{\text{tot}}}{\sum_j e'_j}. \tag{3}$$

Table 9: Bias pattern b_k as a function of Chebyshev distance from the central cell (cell 38).

Distance	b_k	**Region**
0	$+1.0$	Central cell
1	$+0.4$	First ring (8 cells)
2	-0.4	Second ring (16 cells)
≥ 3	-1.0	Outer cells

This pattern enhances the cluster centre and suppresses the tails, producing narrower showers that mimic typical data–MC differences observed in the ATLAS electromagnetic calorimeter.

6.2.5 Correction methods

Cell energy fractions are defined as $f_k = e_k/E_{\text{tot}}$ where $E_{\text{tot}} = \sum_j e_j$.

Method 1 — Flat shift. A single additive correction per cell per $|\eta|$ bin:

$$\Delta_k = \langle f_k^{\text{pseudo}} \rangle - \langle f_k^{\text{MC}} \rangle. \tag{4}$$

Application: $f_k^{\text{corr}} = f_k^{\text{MC}} + \Delta_k$.

Method 2 — Energy-dependent TProfile correction. A `TProfile` records the mean correction as a function of the MC cell fraction:

$$\alpha_k(f) = \Big\langle f_k^{\text{pseudo}} - f_k^{\text{MC}} \Big\rangle \Big|_{f_k^{\text{MC}} = f}. \tag{5}$$

Application: $f_k^{\text{corr}} = f_k^{\text{MC}} + \alpha_k\Big(f_k^{\text{MC}}\Big)$, where α_k is obtained via `TProfile::Interpolate()`.

In both methods, cluster energy is conserved by rescaling:

$$e_k^{\text{corr}} = \frac{f_k^{\text{corr}}}{\sum_j f_j^{\text{corr}}} \cdot E_{\text{tot}}. \tag{6}$$

6.2.6 Quality cuts

Two quality requirements are applied consistently across all pipeline stages:

1. **Negative energy clamping** (`clampCellEnergies()`): Cell energies below zero (from electronic noise subtraction) are set to zero before computing fractions.
2. **Central cell dominance** (`isHealthyCluster()`): The central cell (index 38) must have the highest energy fraction. This rejects mis-centred clusters.

6.2.7 Configuration parameters (`config.h`)

Key constants defined in `config.h`:

Table 10: Configurable parameters in `config.h`.

Constant	Value	Description
`kEtaSize`, `kPhiSize`	7, 11	Cluster dimensions ($\eta \times \phi$)
`kCentralCell`	38	Flat index of the central cell
`kDeltaEta`	0.025	Layer-2 cell η granularity
`kNEtaBins`	14	Number of $\lvert\eta\rvert$ bins
`kEtaLimits[15]`	$\{0, 0.2, \ldots, 2.4\}$	$\lvert\eta\rvert$ bin edges
`kNFracBins`	12	TProfile energy-fraction bins
`kFracBins[13]`	$\{-1, -0.5, \ldots, 0.5, 1\}$	Energy-fraction bin edges
`kTreeName`	`"tree"`	TTree name in ntuple files
`kCellBranch`	`"photon.7x11ClusterLr2E"`	Cell energy branch name
`kEtaBranch`	`"photon_cluster.eta2"`	Cluster η branch name

6.3 Grid submission (ntuple production)

Ntuples are produced from AOD files using the NTupleMaker framework submitted via pathena. A valid grid proxy is required.

```
# Set up grid tools
setupATLAS
asetup Athena,25.0.40
source NTupleMaker_workspace/build/x86_64-el9-gcc14-opt/setup.sh
lsetup panda rucio
voms-proxy-init -voms atlas

# Submit MC23e Z->llgamma jobs
cd NTupleMaker_workspace/run/
bash ShowerShapesAnalysis/grid/submit_mc23e_Zllg.sh
```

The grid scripts automatically set the correct output dataset names, number of files per job, and analysis type (`eegamma` or `mumugamma`).

7 Output

All output is written under the `output/` directory and is gitignored.

7.1 Validation output

Table 11: Output files from the validation workflow.

File	Contents
`output/reta_Zeeg.root`	TH1F histograms: `h_reta_fudged`, `h_reta_unfudged`, `h_reta_computed`, difference histograms, and a 2D correlation histogram.
`output/rphi_Zeeg.root`	Same structure for R_ϕ.
`output/weta2_Zeeg.root`	Same structure for $w_{\eta 2}$.
`output/plots/reta_Zeeg.pdf`	Single-page PDF: overlay + ratio plot for R_η.
`output/plots/rphi_Zeeg.pdf`	Same for R_ϕ.
`output/plots/weta2_Zeeg.pdf`	Same for $w_{\eta 2}$.

Identical files are produced for each sample (`Zeeg`, `Zmumug`).

7.2 Closure test output

For each distortion level, a directory is created:

```
output/closure_test_<N>pct/      # N = 1, 3, or 5
```

Table 12: Output files per distortion level.

File	Contents
`pseudo.root`	TTree with distorted cell energies (pseudo-data).
`corrections.root`	Method 1: TH2D per $\|\eta\|$ bin (`flatShift_eta_X.XX_X.XX`). Method 2: 77 TProfiles per $\|\eta\|$ bin (`CellCorrection_Cell_K_Inc_Eta_X.XX_X.XX`).
`reweighted_m1.root`	TTree with Method 1 corrected cell energies.
`reweighted_m2.root`	TTree with Method 2 corrected cell energies.
`closure_histos.root`	TH1D histograms per $\|\eta\|$ bin per variable per source (MC, pseudo, M1, M2). Naming: `h_<var>_<source>_eta<N>`.
`closure_reta.pdf`	Multi-page PDF (13 pages): R_η closure plots, one page per populated $\|\eta\|$ bin.
`closure_rphi.pdf`	Same for R_ϕ.
`closure_weta2.pdf`	Same for $w_{\eta 2}$.

Histogram naming convention. Within `closure_histos.root`, histograms follow the pattern:

```
h_<variable>_<source>_eta<bin>
```

where `<variable>` $\in$ {`reta`, `rphi`, `weta2`}, `<source>` $\in$ {`mc`, `pseudo`, `m1`, `m2`}, and `<bin>` $\in [0, 13]$.

Plot format. Each closure plot page contains:

- **Upper panel**: Normalised overlay of pseudo-data (black markers), MC (red filled), Method 1 (green dashed), Method 2 (blue solid).
- **Lower panel**: Ratio to pseudo-data (range 0.8–1.2).
- **Text labels**: $|\eta|$ bin range, distortion level.

8 Reproducibility

8.1 Full reproduction from scratch

To reproduce the complete set of results, execute the following commands in order:

```
# 1. Set up environment
setupATLAS
lsetup "root 6.32.08-x86_64-el9-gcc13-opt"

# 2. Navigate to the repository
cd /path/to/ShowerShapesAnalysis

# 3. Ensure input ntuple is present
ls -lh ntuples/mc23e/mc23e_700770_Zeeg.root
# Expected: ~13 GB, ~9.6M events

# 4. Run validation (all 3 variables, both channels)
cd scripts
mkdir -p ../output/plots

python compute_reta.py   -i ../ntuples/mc23e/mc23e_700770_Zeeg.
   root \
                          -o ../output/reta_Zeeg.root
python compute_rphi.py   -i ../ntuples/mc23e/mc23e_700770_Zeeg.
   root \
                          -o ../output/rphi_Zeeg.root
python compute_weta_2.py -i ../ntuples/mc23e/mc23e_700770_Zeeg.
   root \
                          -o ../output/weta2_Zeeg.root

python compute_reta.py   -i ../ntuples/mc23e/mc23e_700771_Zmumug
   .root \
                          -o ../output/reta_Zmumug.root
python compute_rphi.py   -i ../ntuples/mc23e/mc23e_700771_Zmumug
   .root \
                          -o ../output/rphi_Zmumug.root
python compute_weta_2.py -i ../ntuples/mc23e/mc23e_700771_Zmumug
   .root \
                          -o ../output/weta2_Zmumug.root

root -l -b -q 'plot_reta.C("../output/reta_Zeeg.root",\
  "../output/plots/reta_Zeeg")'
```

```
root -l -b -q 'plot_rphi.C("../output/rphi_Zeeg.root",\
  "../output/plots/rphi_Zeeg")'
root -l -b -q 'plot_weta_2.C("../output/weta2_Zeeg.root",\
  "../output/plots/weta2_Zeeg")'

root -l -b -q 'plot_reta.C("../output/reta_Zmumug.root",\
  "../output/plots/reta_Zmumug")'
root -l -b -q 'plot_rphi.C("../output/rphi_Zmumug.root",\
  "../output/plots/rphi_Zmumug")'
root -l -b -q 'plot_weta_2.C("../output/weta2_Zmumug.root",\
  "../output/plots/weta2_Zmumug")'

# 5. Run closure test suite (all 3 distortion levels)
cd closure_test
bash run_closure_suite.sh -1     # -1 = all events (~9.6M)
```

8.2 Expected runtime

Table 13: Approximate runtime on lxplus (single core).

Task	Time
Validation (1 variable, 1 sample, 9.6M events)	∼15 min
Validation (all 3 variables, 2 samples)	∼90 min
Closure test (1 level, 9.6M events, all 6 steps)	∼60 min
Closure test suite (3 levels, 9.6M events)	∼3 hours
Closure test (500k events, quick test)	∼5 min

8.3 Random seeds

The pseudo-data generation uses `TRandom3` with seed 0 (which initialises from the system clock UUID). This means that pseudo-data files are *not* bitwise-reproducible across runs by default. To obtain deterministic results, modify `create_pseudodata.C` to use a fixed seed:

```
TRandom3 rng(42);  // Replace rng(0) with fixed seed
```

The χ^2 closure metrics are stable across random realisations because they are averaged over ∼7.8M events (after quality cuts).

8.4 Version control

The repository tracks only source code and configuration. All input data (ntuples), output files (histograms, plots, ROOT files), and compiled artifacts are excluded via `.gitignore`.

Key gitignore rules:

```
ntuples/            # Input ntuples (large)
output/             # All computed outputs
*.root              # ROOT files in top directory
```

```
__pycache__/        # Python bytecode
*.log               # Log files
```

Recommended commit workflow.

```
# Check what changed
git status
git diff --stat

# Stage and commit with conventional prefix
git add -A
git commit -m "feat: add new shower shape variable"
git push origin main
```

Use conventional commit prefixes: `feat:` (new feature), `fix:` (bug fix), `docs:` (documentation), `chore:` (maintenance), `refactor:` (restructuring).